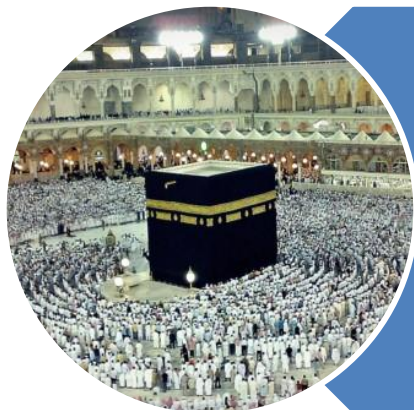


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Linked List

Operations

C

P

C

S

2

0

4

3

Operations

- Is Empty
- Traversing
- Modifying
- Searching
- Insertion
- Deletion
- Merging

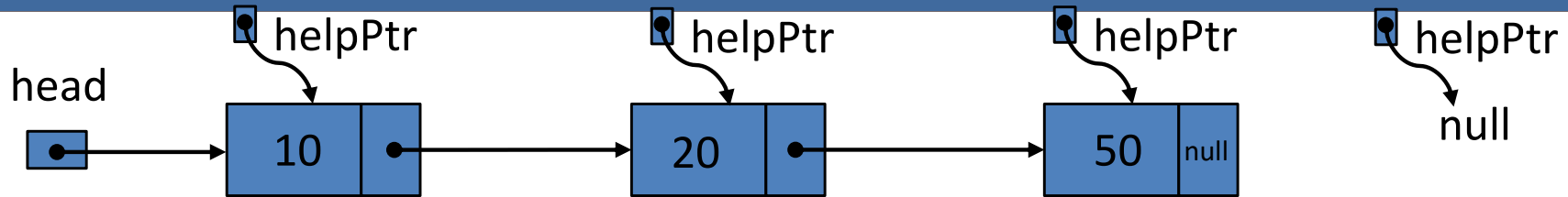
Linked List

Operations

(Searching)

"No Bonus" marks for this course anymore

C
P
C
S
2
0
4



- Searching a key value
- Head points to first node

Return false

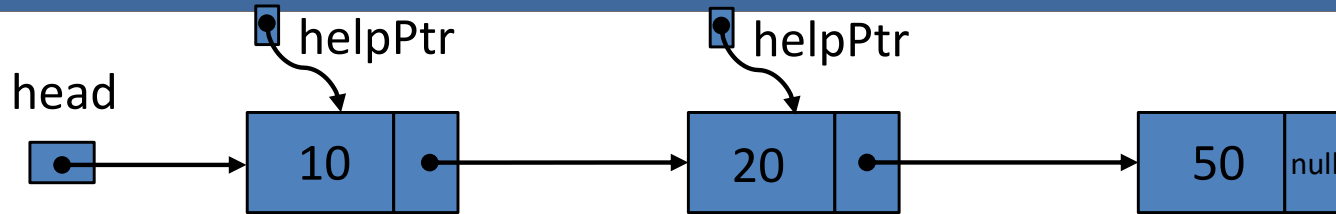
```

public boolean searchNode(int value) {
    LLnode helpPtr = head;
    while (helpPtr != null) {
        if (helpPtr . data == value;
            return true;
        helpPtr = helpPtr.next;
    }
    return false;
}
  
```

Linked List – Searching 60

"No Bonus" marks for this course anymore

C
P
C
S
2
0
4



- Searching a key value
- Head points to first node

Return true

```

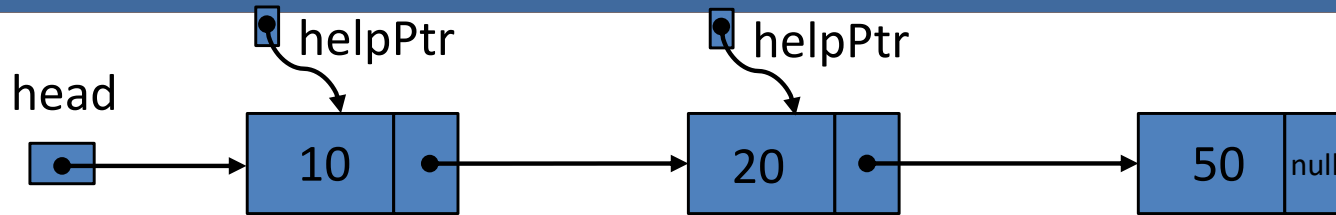
public boolean searchNode(int value) {
    LLnode helpPtr = head;
    while (helpPtr != null) {
        if (helpPtr . data == value;
            return true;
        helpPtr = helpPtr.next;
    }
    return false;
}
  
```

Linked List – Searching 20

“No Bonus” marks for this course anymore

C
P
C
S

2
0
4



- Searching a key value
- Head points to first node

Return helpPtr
(address of 20)

```

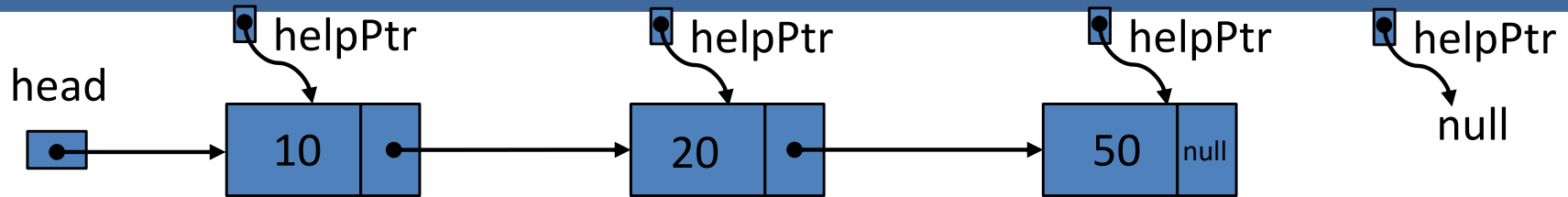
public LLnode searchNode(int value) {
    LLnode helpPtr = head;
    while (helpPtr != null) {
        if (helpPtr . data == value;
            return helpPtr;
        helpPtr = helpPtr.next;
    }
    return helpPtr;
}

```

Linked List – Searching 20

"No Bonus" marks for this course anymore

C
P
C
S
2
0
4



- Searching a key value
- Head points to first node

Return helpPtr
(null)

```

public LLnode searchNode(int value) {
    LLnode helpPtr = head;
    while (helpPtr != null) {
        if (helpPtr . data == value;
            return helpPtr;
        helpPtr = helpPtr.next;
    }
    return helpPtr;
}
  
```

Linked List – Searching 70

Linked List

Operations

(Insertion)

C

P

C

S

2

0

4

Linked List – Insertion

- 4 – Cases
 - List is **empty**
 - Insert as a **first node**
 - Insert as a **last node**
 - Anywhere **between** two nodes
- Assumption
 - Data of linked list is always **“SORTED”**

C

P

C

S

2

0

4

Insertion— **Empty list**

- Check list is empty?

- `head == null`



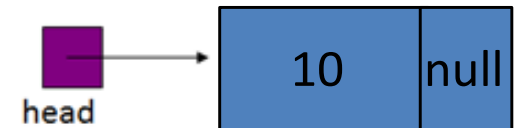
- Create new node

- `new LNode(value)` OR
 - `new LNode(value, null)` OR
 - `new LNode(value, head)`



- Make head point to new memory

- `head = new LNode(value)` OR
 - `head = new LNode(value, head)`



C

P

C

S

2

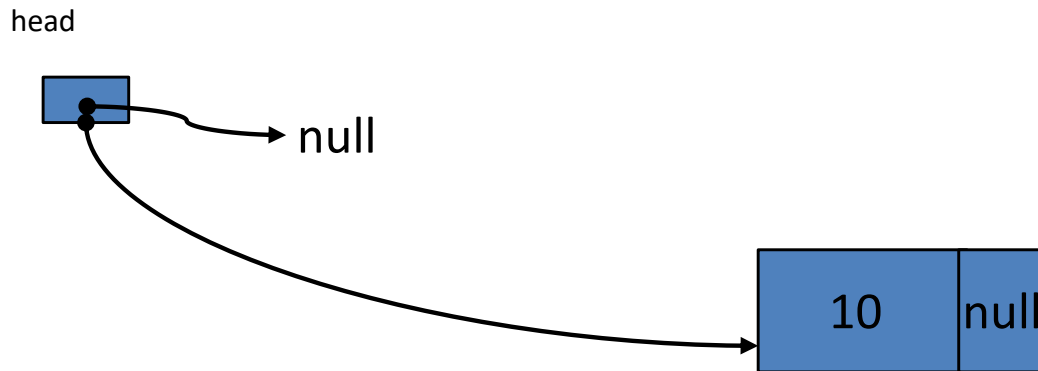
0

4

12

Linked List – Insertion (Empty)

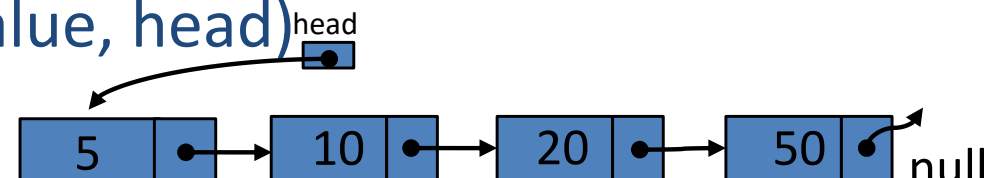
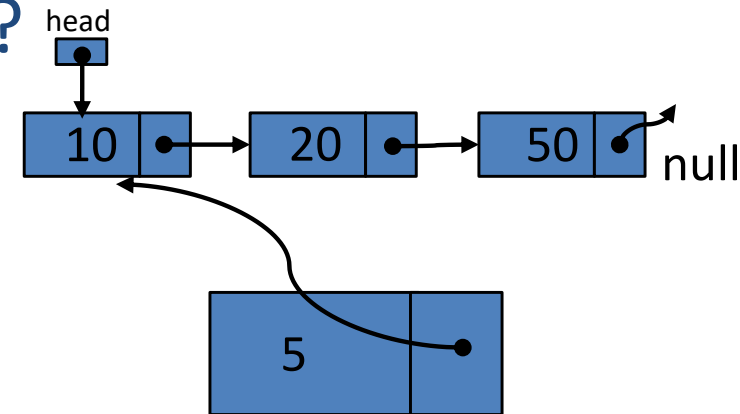
```
public void insertEmptylist(int value) {
    // Insertion Empty
    if (head == null) {
        head = new LLnode(value, head)
    }
}
```



- **Value = 10**

Insertion— as First node

- Can value be inserted as first?
 - $\text{head.data} > \text{value}$
- Create new node
 - **new** LNode(value, head)
- Make head point to new memory
 - **head** = new LNode(value, head)



C

P

C

S

2

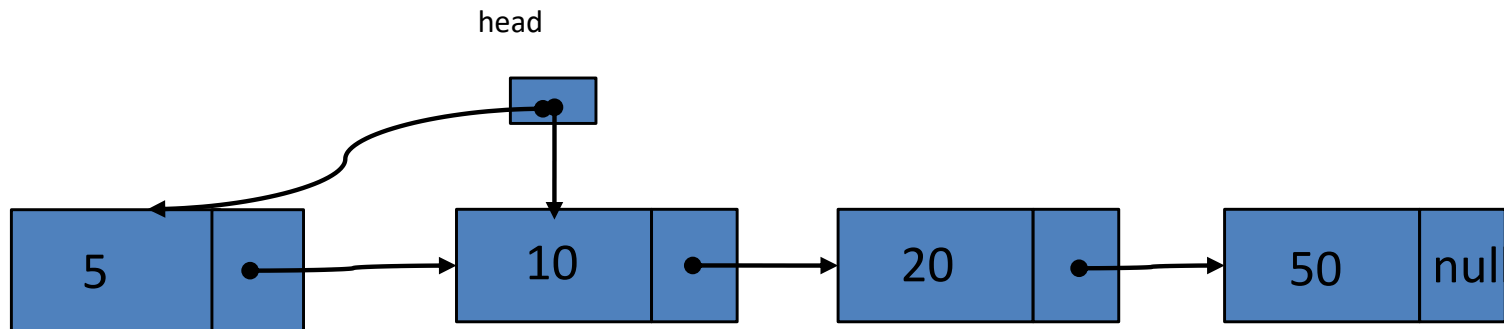
0

4

14

Linked List – Insertion (First)

```
public void insertFirst(int value) {
    // Insertion as First node for non-empty list
    if (head.data > value) {
        head = new LLnode(value, head)
    }
}
```

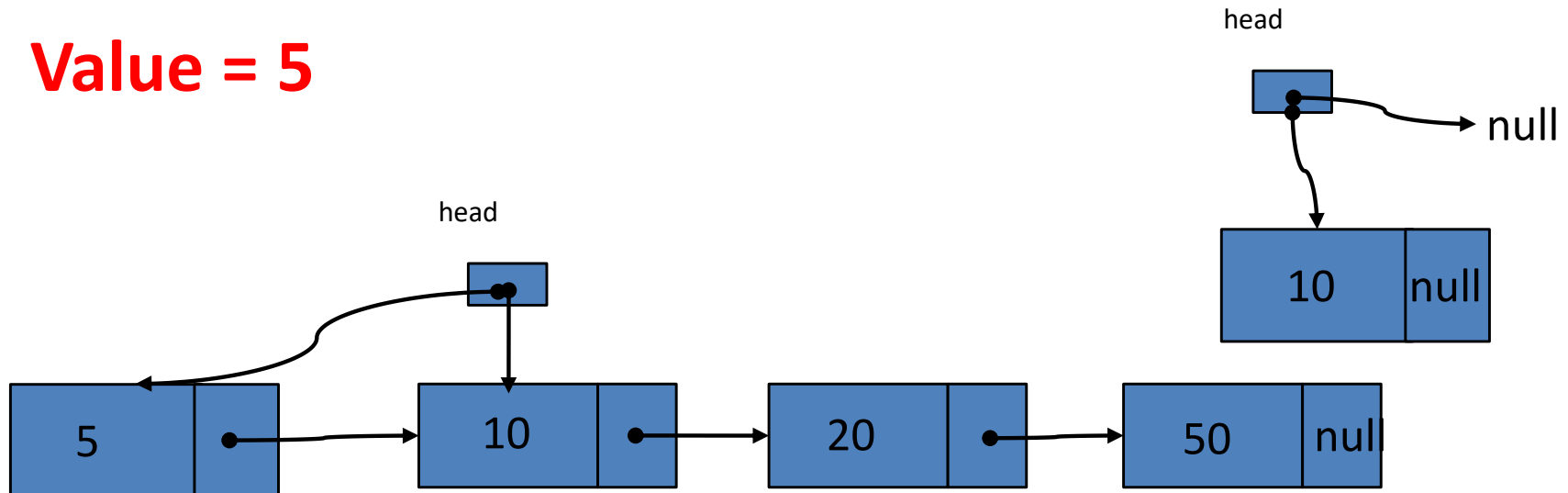


- **Value = 5**

Linked List – Insertion (First & Empty)

```
public void insert(int value) {
    // Insertion as First node, Empty or non-empty list
    if (head == null || head.data > value) {
        head = new LLnode(value, head)
    }
}
```

Value = 5



C

P

C

S

2

0

4

Insertion— as Last node

- Traverse to the last node

- `helpPtr = head`

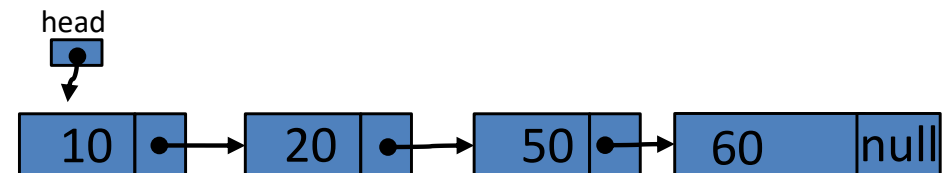
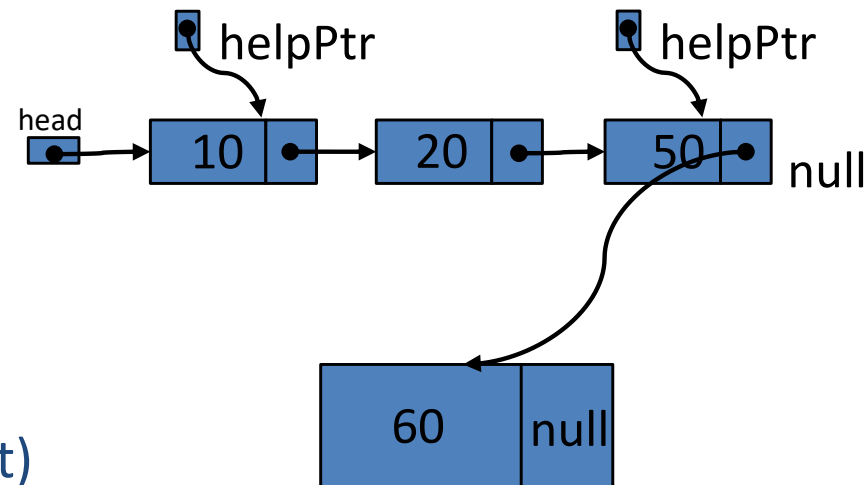
- `helpPtr.next != null`

- Create new node

- `new LLnode(value, helpPtr.next)`

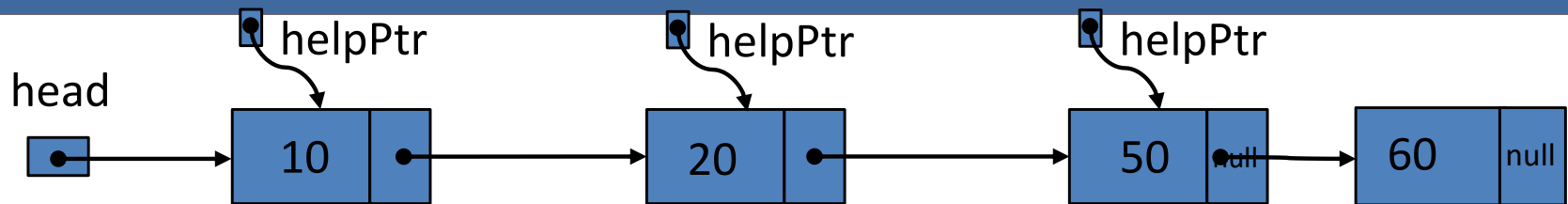
- Make `helpPtr.next` point to new memory

- `helpPtr.next = new LLnode(value, helpPtr.next)`



Insertion – as Last node 60

C
P
C
S
2
0
4



```
public void insertLast(int value) {
    // Insertion as First node, Empty or non-empty list
    if (head == null || head.data > value) {
        head = new LLnode(value, head)
    }
    // Insertion anywhere else even at last
    else {
        LLnode helpPtr = head;
        while (helpPtr.next != null) {
            helpPtr = helpPtr.next;
        }
        // Now make new node and insert
        helpPtr.next = new LLnode(value, helpPtr.next);
    }
}
```

C

P

C

S

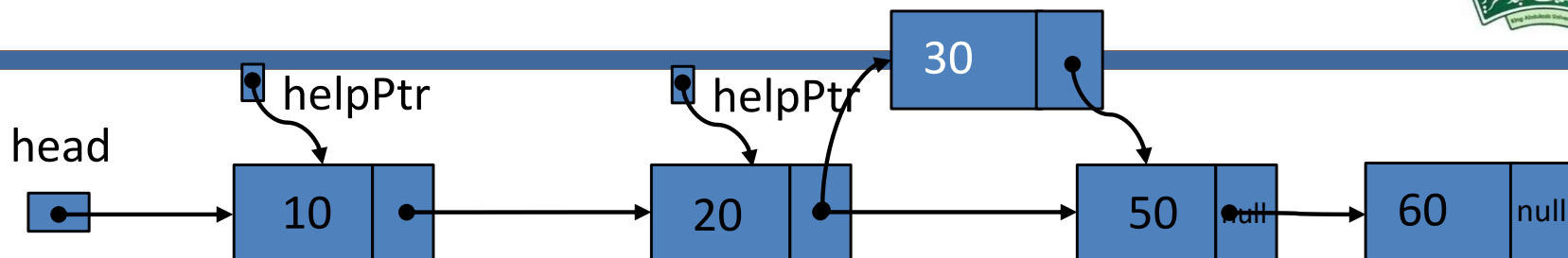
2

0

4

Insertion – **in middle**

- Find the location of the new node's **predecessor**
- Create **new node**
- Point the new node to the **successor**
- Point the **predecessor** node to the new node



```
public void insertMiddle(int value) {
    // Insertion as First node, Empty or non-empty list
    if (head == null || head.data > value) {
        head = new LLnode(value, head)
    }
    else {
        LLnode helpPtr = head;
        while (helpPtr.next != null) {
            if (helpPtr.next.data > value)
                break; // stop traversal
            helpPtr = helpPtr.next;
        }
        helpPtr.next = new LLnode(value, helpPtr.next);
    }
}
```

C

P

C

S

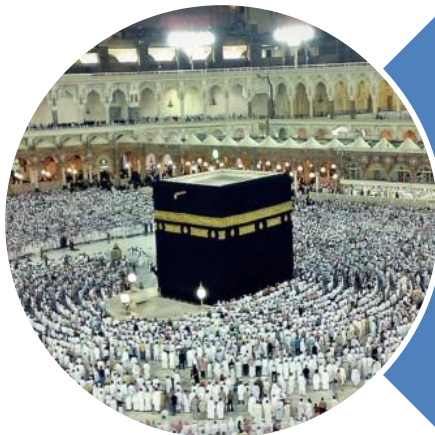
2

0

4

Linked List – **Insertion** (Complete Code)

```
public void insert(int value) {  
    // Insertion as First node, Empty or non-empty list  
    if (head == null || head.data > value) {  
        head = new LLnode(value, head)  
    }  
    // Insertion anywhere else even at last  
    else {  
        LLnode helpPtr = head;  
        while (helpPtr.next != null) {  
            if (helpPtr.next.data > value)  
                break; // we found our spot and stop traversal  
            helpPtr = helpPtr.next;  
        }  
        // Now make new node and insert  
        helpPtr.next = new LLnode(value, helpPtr.next);  
    }  
}
```



Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). **(Quran 3 : 173)**